SSL (Secure Sockets Layer) et TLS (Transport Layer Security)

Emmanuel Conchon
(emmanuel.conchon@unilim.fr)
Supports conçus avec la participation de Cristina ONETE
Université de Limoges

Généralités

- SSL/TLS est une brique essentielle des communications réseaux sécurisées et de la sécurité sur Internet
  - HTTPS inventé par Netscape en 1995 en est une des premières illustrations
  - SSMTP, SPOP3, IMAPS pour les mails
  - VPN SSL
  - EAP SSL
- Elle fonctionne au-dessus de la couche transport (TCP) et est compatible avec quasiment toutes les applications existantes
- Elle assure
  - l'authentification des parties
  - la confidentialité des échanges
  - l'intégrité des données

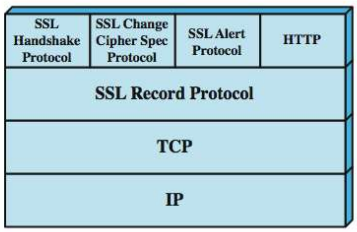
Généralités

- Un peu d'histoire:
  - SSL V1.0: jamais utilisé
  - SSL V3.0 sortie en 1996 (Il ne faut plus utiliser la v2: RFC 6176 de 2011)
  - Intégré dans les navigateurs depuis les années 90
- Deux grandes phases:
  - La connexion SSL
    - Une connexion TCP sur une session existante
    - Fonctionne en point à point de manière transitoire (par session)
  - La session TLS
    - C'est une association créée entre un serveur et un client
    - Définit les paramètres de sécurité à utiliser associés à une session
    - La session évite de renégocier les paramètres à chaque connexion (ex HTTP)
    - En théorie on peut avoir plusieurs sessions en parallèle mais ce mode n'est pas implémenté en pratique



Généralités

- TLS doit être vu comme une évolution de SSL
- TLS (Transport Layer Security)
  - Version standardisée par l'IETF de SSL
  - TLS v1,0 en 1999
  - TLS v1.1 en 2006
  - TLS v1.2 en 2008
  - TLS v1.3 en 2018



- TLS utilise des suites cryptographiques composées des trois fonctionnalités
  - Confidentialité: Algo symétrique (3DES, AES, IDEA, …)
  - Intégrité: HMAC: Hash Message Authentication Code
  - Authentification: X.509 et MAC



Généralités : de TLS 1.2 vers TLS 1.3

- SSL et TLS ont suivi, pendant 20 ans la même structure (jusqu'à TLS 1.2 inc.)
  - Mêmes méthodes d'échange et dérivation de clé, des algorithmes problématiques retenus souvent trop longtemps (MD5)
- A partir de 2013 : un très grand nombre d'attaques publiées contre TLS 1.2 :
  - SMACK, 3Shake : qui exploitent la dérivation de clé
  - FREAK et LogJam : qui exploitent des tailles de clés petites, dites "d'export"
  - DROWN, downgrade attacks : qui exploitent la compatibilité avec des versions vulnérables
  - Attaque par collision de transcript, attaque contre l'authentification : qui exploitent la dérivation de clé
- TLS 1.3 : fondamentalement différent, tente de réparer les failles de TLS 1.2



TLS: Fonctionnement général

- TLS repose essentiellement sur deux protocoles
  - Le protocole TLS Handshake
    - Authentifie le serveur (parfois le client) lors de l'initiation de la connexion
    - Négocie entre le client et le serveur l'algorithme de chiffrement et les clés de session à utiliser
    - Permet la dérivation de clés
  - Le protocole TLS Record
    - Sécurise la connexion en utilisant les algorithmes négociés
    - Vérifie l'intégrité des données à l'aide de HMAC
    - Peut faire évoluer les clés calculées lors du Handshake
    - Permet l'envoi d'un message spécial (dit ticket) pour résumer des sessions



La session TLS

- A chaque fois qu'une session est ouverte, TLS met en place un état associé
  - Sans état, aucun trafic ne peut circuler
- La mise en place de la session peut s'illustrer de la manière suivante
  - Demande d'ouverture de connexion
  - Initialisation avec les flux utilisateurs bloqués et un état vide
  - TLS Handshake
  - Activation de l'état négocié
  - Autorisation des flux utilisateurs



Le protocole TLS Handshake dans TLS 1.2

- Ce protocole est le plus complexe de TLS et permet aux deux parties de s'authentifier de manière réciproque
- Le protocole se découpe en fait en 4 phases
  - L'Initiation de connexion
  - L'échange de certificat et de clés
  - La vérification du certificat par le client et le paramétrage de la connexion
  - La mise en place finale de la connexion sécurisée
- Ce protocole a été pensé pour résister
  - Aux attaques par abaissement de version (rollback)
  - Aux attaques par rejeu



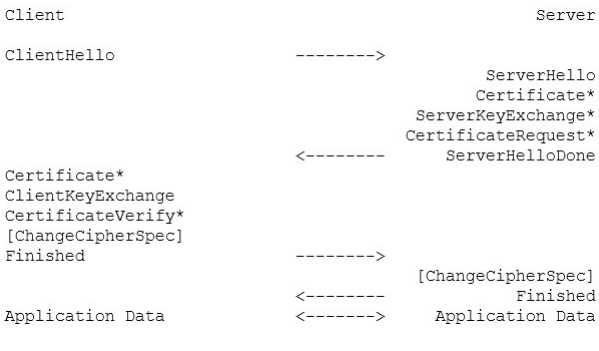
Le principe de session résumée

- Une paire client/serveur peut communiquer à plusieurs reprises :
  - Par exemple, vous pouvez aller sur un même site web plusieurs fois par jour
- Si le client et serveur se connaissent, ils peuvent "résumer" leur session :
  - Un Handshake abrégé
  - Authentification par connaissance d'une ou des clés antérieurement calculées
  - Les nouvelles clés dépendront d'un secret antérieurement dérivé (PSK)

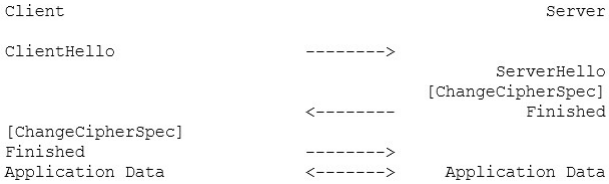


Le protocole TLS Handshake dans TLS 1.2

- Echanges de messages lors d'un handshake complet en TLS 1.2 (tiré de la RFC)

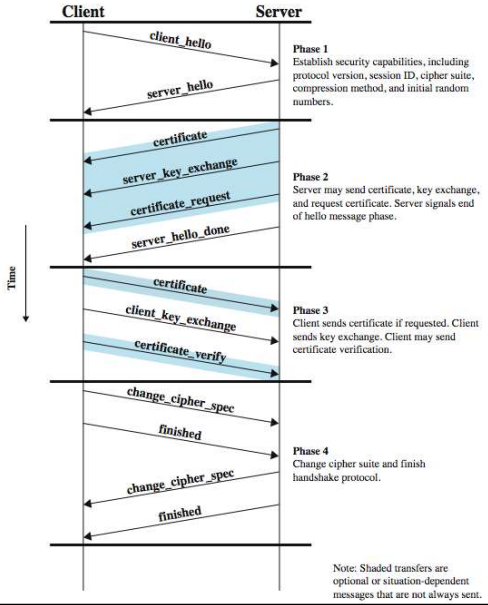


- Echanges lors d'un handshake réduit





Le protocole TLS Handshake dans TLS 1.2



Le protocole TLS Handshake dans TLS 1.2

- Le message Client Hello
  - Ce message émis par le client va contenir
    - La version de TLS à utiliser
    - Les suites cryptographiques que le client peut mettre en œuvre
    - Le mécanisme de compression à appliquer aux données
    - Un champ random contenant un aléa utilisé lors de la dérivation des clés
- La réponse du serveur peut être de deux types
  - Un message de type Alert si aucune des propositions du client n'est satisfaisante
  - Un message Serveur Hello contenant:
    - Le choix de la suite cryptographique retenue
    - Une valeur aléatoire serveur_random
  - En fonction du choix de la suite, cette réponse peut être suivie d'un certificat

Le protocole TLS Handshake dans TLS 1.2

- Les suites cryptographiques négociées lors de cette première étape sont cruciales pour la définition du niveau de sécurité attendu
  - Elles se décrivent sous la forme: TLS_KE_WITH_CYPHER_HASH
    - Dans la version 1,2 il est proposé de faire évoluer cette notation sous la forme: TLS_KEA_SIGN_WITH_CIPHER_HASH
  - Par exemple, TLS_RSA_WITH_AES_256_CBC_SHA indique que:
    - RSA sera utilisé pour l'authentification du serveur et pour établir les clés de session
    - AES 256 CBC sera utilisé comme algorithme de chiffrement
    - SHA indique qu'un HMAC SHA-1 sera utilisé pour assurer l'intégrité du canal de communication
  - Toutes les suites sont définies dans la RFC 5246 pour TLS 1.2
    - Support de DH ou de RSA pour l'échange de clés
      - ★ *DH/ECDHE assure le PFS (Perfect Forward Secrecy)*
        - La découverte de la clé privée ne compromet pas la confidentialité des échanges passés



Le protocole TLS Handshake : server key-exchange

- Le message Server Key Exchange
  - Démarre la phase d'échange de clé après la négociation des paramètres
    - Deux possibilités : échange de clé basé sur RSA ou via Diffie-Hellman
  - Représente la contribution du serveur à l'échange de clé
  - TLS-RSA :
    - l'élément est vide, étant contenu dans le prochain message (le certificat)
    - le serveur contribue une clé publique de chiffrement RSA
  - TLS-DH :
    - élément vide (compris dans le certificat)
    - le serveur contribue un élément Diffie Hellman (de type g^b)
  - TLS-DHE :
    - élément non-vide
    - contient un triplet (p, g, g^b) décrivant un groupe et un élément Diffie Hellman



Le protocole au sein de TLS-DH




Choisir $a \in \{1, \dots, q-1\}$ Choisir $b \in \{1, \dots, q-1\}$

g^b ←

→ g^a

Calcule $K = (g^b)^a$ Calcule $K = (g^a)^b$

Seulement sécurisé contre des attaquants passifs

- Pour sécuriser les échanges contre un attaquant actif, en TLS on authentifie la contribution du serveur :
  - DH : g^b certifiée
  - DHE : nouveau choix de (p, g, g^b) à chaque session, signé avec une clé certifiée



Le protocole au sein de TLS-RSA




Choisir K aléatoirement

$\text{c} = \text{RSA. Enc}(pk_B; K)$ →

(sk_B, pk_B)

$K = \text{RSA. Dec}(sk_B; c)$

- Pour sécuriser les échanges contre un attaquant actif, en TLS on certifie la clé RSA du serveur
  - Ainsi nous sommes sûrs que seulement le serveur pourra déchiffrer avec la clé



Le protocole TLS Handshake

Key exchange/agreement and authentication

Source: wikipedia

Algorithm	SSL 2.0	SSL 3.0	TLS 1.0	TLS 1.1	TLS 1.2	TLS 1.3 (Draft)	Status
RSA	Yes	Yes	Yes	Yes	Yes	No	Defined for TLS 1.2 in RFCs
DH-RSA	No	Yes	Yes	Yes	Yes	No	
DHE-RSA (*forward secrecy*)	No	Yes	Yes	Yes	Yes	Yes	
ECDH-RSA	No	No	Yes	Yes	Yes	No	
ECDHE-RSA (*forward secrecy*)	No	No	Yes	Yes	Yes	Yes	
DH-DSS	No	Yes	Yes	Yes	Yes	No	
DHE-DSS (*forward secrecy*)	No	Yes	Yes	Yes	Yes	No[23]	
ECDH-ECDSA	No	No	Yes	Yes	Yes	No	
ECDHE-ECDSA (*forward secrecy*)	No	No	Yes	Yes	Yes	Yes	
PSK	No	No	Yes	Yes	Yes		
PSK-RSA	No	No	Yes	Yes	Yes		
DHE-PSK (*forward secrecy*)	No	No	Yes	Yes	Yes		
ECDHE-PSK (*forward secrecy*)	No	No	Yes	Yes	Yes		
SRP	No	No	Yes	Yes	Yes		
SRP-DSS	No	No	Yes	Yes	Yes		
SRP-RSA	No	No	Yes	Yes	Yes		
Kerberos	No	No	Yes	Yes	Yes		
DH-ANON (*insecure*)	No	Yes	Yes	Yes	Yes		
ECDH-ANON (*insecure*)	No	No	Yes	Yes	Yes		
GOST R 34.10-94 / 34.10-2001[24]	No	No	Yes	Yes	Yes		Proposed in RFC drafts



Le protocole TLS Handshake

Source:wikipedia

Cipher security against publicly known feasible attacks

Cipher			Protocol version						Status
Type	Algorithm	Nominal strength (bits)	SSL 2.0	SSL 3.0 [n 1][n 2][n 3][n 4]	TLS 1.0 [n 1][n 3]	TLS 1.1 [n 1]	TLS 1.2 [n 1]	TLS 1.3 (Draft)	
Block cipher with mode of operation	AES GCM[25][n 5]	256, 128	N/A	N/A	N/A	N/A	Secure	Secure	Defined for TLS 1.2 in RFCs
	AES CCM[26][n 5]		N/A	N/A	N/A	N/A	Secure	Secure	
	AES CBC[n 6]		N/A	N/A	Depends on mitigations	Secure	Secure	N/A	
	Camellia GCM[27][n 5]	256, 128	N/A	N/A	N/A	N/A	Secure	Secure	
	Camellia CBC[28][n 6]		N/A	N/A	Depends on mitigations	Secure	Secure	N/A	
	ARIA GCM[29][n 5]	256, 128	N/A	N/A	N/A	N/A	Secure	Secure	
	ARIA CBC[29][n 6]		N/A	N/A	Depends on mitigations	Secure	Secure	N/A	
	SEED CBC[30][n 6]	128	N/A	N/A	Depends on mitigations	Secure	Secure	N/A	
	3DES EDE CBC[n 6]	112[n 7]	Insecure	Insecure	Insecure	Insecure	Insecure	N/A	
	GOST 28147-89 CNT[24]	256	N/A	N/A	Secure	Secure	Secure		Defined in RFC 4357
	IDEA CBC[n 6][n 8]	128	Insecure	Insecure	Insecure	Insecure	N/A	N/A	Removed from TLS 1.2
	DES CBC[n 6][n 8]	56	Insecure	Insecure	Insecure	Insecure	N/A	N/A	
		40[n 9]	Insecure	Insecure	Insecure	N/A	N/A	N/A	Forbidden in TLS 1.1 and later
	RC2 CBC[n 6]	40[n 9]	Insecure	Insecure	Insecure	N/A	N/A	N/A	
Stream cipher	ChaCha20-Poly1305[34][n 5]	256	N/A	N/A	N/A	N/A	Secure	Secure	Defined for TLS 1.2 in RFCs
	RC4[n 10]	128	Insecure	Insecure	Insecure	Insecure	Insecure	N/A	Prohibited in all versions of TLS by RFC 7465
		40[n 9]	Insecure	Insecure	Insecure	N/A	N/A	N/A	
None	Null[n 11]	-	N/A	Insecure	Insecure	Insecure	Insecure	Insecure	Defined for TLS 1.2 in RFCs



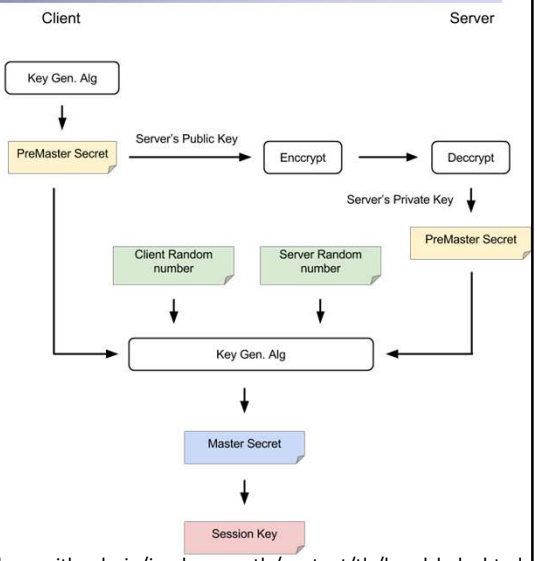
Le protocole TLS Handshake

- Le message Certificate
  - Il se compose d'une suite de certificats (X.509 en général)
    - Le premier certificat désigne le serveur TLS
    - Les autres certificats composants la chaine servent pour vérifier la chaine de confiance
      - ★ *Le certificat de l'autorité racine peut être omis*
    - En pratique, ces chaines de certificats sont souvent mal gérées au niveau des implémentations
      - ★ *Certificats en double, désordonnés, chaine incomplète….*
  - Ce message peut également être transmis par le client si une authentification de celui-ci est demandée par le serveur



Le protocole TLS Handshake

- Le message Client_key_exchange
  - Une fois la phase d'authentification terminée, le client utilise ce message pour transmettre la clé pre-master-secret (PMSK)
  - Ce message est chiffré à l'aide de la clé publique du serveur
  - Cette clé est utilisée avec les deux aléas pour produire la master secret key d'une longueur de 48 octets
    - Une fois cette clé générée, la pre-master-secret est détruite



Source: https://chunminchang.gitbooks.io/j-pake-over-tls/content/tls/handshake.html



Le protocole TLS Handshake : client key-exchange

- Le message Client_key_exchange
  - L'équivalent, côté client, du message server key-exchange
  - TLS-RSA :
    - Le client choisit une valeur aléatoire pms (qui servira comme clé intermédiaire)
    - Le client envoie un chiffrement de pms avec la clé publique du serveur
  - TLS-DH :
    - le client contribue un élément Diffie Hellman (de type g^a) dans le groupe indiqué dans le certificat du serveur
  - TLS-DHE :
    - le client contribue un élément Diffie Hellman (de type g^a) dans le groupe indiqué dans le message de server key-exchange



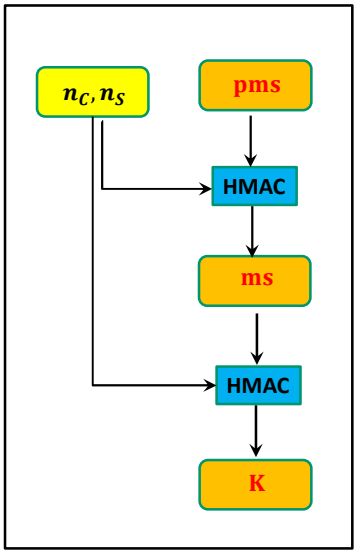
Le protocole TLS handshake : les propriétés

- Les messages client key-exchange et server key-exchange permettent aux deux parties de calculer le secret pms (pre-master secret)
  - En RSA le client choisit et encapsule cette valeur pour la transférer
  - En DH/DHE, le client et serveur calculent $pms = g^{ab}$
  - Ce secret sera ensuite utilisé pour dériver des clés cryptographiques
- Le but est d'obtenir une valeur de pms indistinguable d'un élément aléatoire du même groupe
- De plus, nous voulons la propriété *perfect forward secrecy* :
  - Assure la sécurité des sessions terminées avant d'une corruption de la clé privée du serveur
  - Cette propriété est seulement garantie



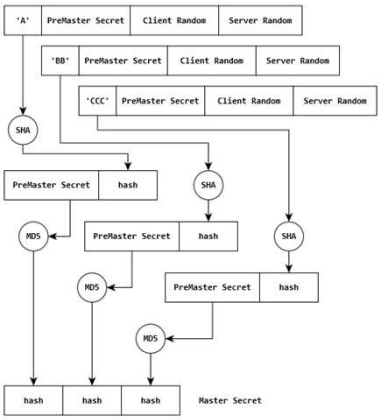
Le protocole TLS Handshake : la dérivation de clés

- Tout commence avec la valeur pms
- Ensuite, le pms est utilisé avec les deux aléas pour produire le master secret d'une longueur de 48 octets
  - Une fois cette clé générée, la pre-master-secret est détruite
- A partir de ms, avec les deux aléas, on obtient les clés de session



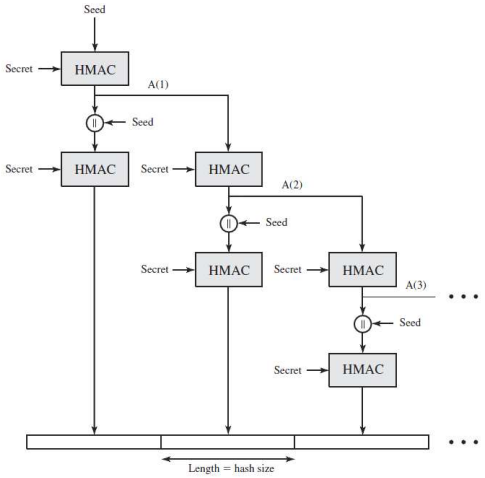
Le protocole TLS Handshake

- A partir de la Master Secret Key (MSK), les clés de session sont générées par une Pseudo Random Function (PRF)
  - Dans TLS 1.1, cette PRF était basée sur une combinaison MD5/SHA-1



Le protocole TLS Handshake

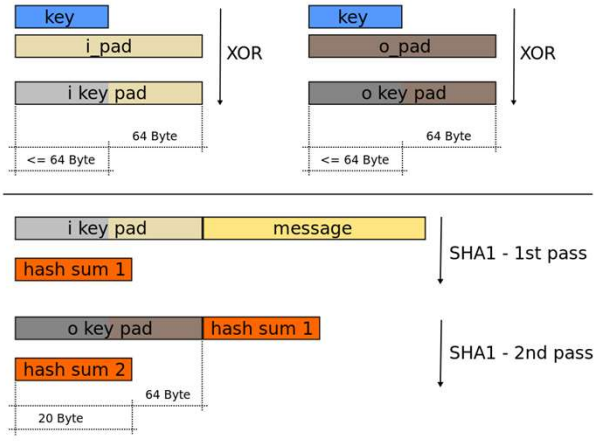
- Depuis TLS 1.2, la PRF est définie dans chaque suite cryptographique et on parle de TLS Hash



- Elle repose sur l'utilisation de KDF (Key Derivation Function): HMAC, HKDF

Chiffrement authentifié (MAC) - HMAC

- Dans TLS 1.2, TLS Hash repose essentiellement sur la fonction HMAC
  - Exemple avec HMAC-SHA1



TLS Hash - HMAC

- Ou de manière générale

$$\mathrm{HMAC}(K, m) = \mathrm{H}\left((K' \oplus opad) \parallel \mathrm{H}\left((K' \oplus ipad) \parallel m \right) \right)$$

$$K' = \begin{cases} \mathrm{H}(K) & K \text{ is larger than block size} \\ K & \text{otherwise} \end{cases}$$

where

H is a cryptographic hash function

m is the message to be authenticated

K is the secret key

K' is a block-sized key derived from the secret key, K; either by padding to the right with 0s up to the block size, or by hashing down to less than or equal to the block size first and then padding to the right with zeros

$\parallel$ denotes concatenation

$\oplus$ denotes bitwise exclusive or (XOR)

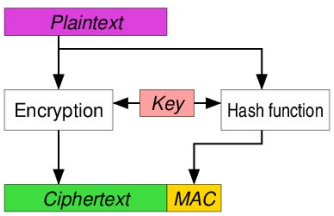
opad is the block-sized outer padding, consisting of repeated bytes valued 0x5c

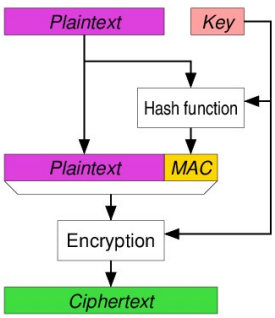
ipad is the block-sized inner padding, consisting of repeated bytes valued 0x36



Rappels sur MAC + Chiffrement

- Il existe trois modes d'utilisation
  - Encrypt & MAC
    - Utilisé dans SSH
  - MAC then Encrypt
    - Utilisé dans TLS 1.2 mais vulnérable à certaines attaques comme Lucky Thirteen





- Le troisième plus tard ☺



HKDF: Extract the Expand

- HKDF est une KDF basée sur HMAC et repose sur deux phases
  - Extract: génère à partir des paramètres d'entré, une clé pseudo-aléatoire de taille fixe
  - Expand: découpe la clé précédente en plusieurs sous clés



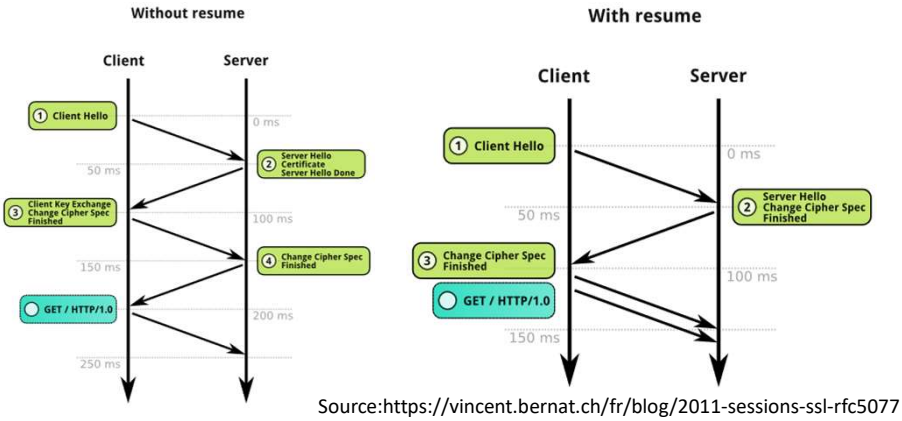
Les sessions et connexions TLS

- Chaque clé générée par la PRF va permettre de sécuriser une session TLS
  - Pour rappel, une session TLS correspond à une communication bi-directionnelle entre un client et un serveur
  - Chaque message échangé dans une session est appelé un record et est attaché à une connexion
- Pour chaque connexion, des états sont maintenus comme pour les sessions
  - Un état va définir l'environnement d'exécution d'un record
  - Il se définit par:
    - Extrémité (client ou serveur)
    - PRF utilisées pour générer les clés
    - Algorithme de chiffrement, algorithme MAC et algorithme de compression (+paramètres)
    - MSK
    - Aléa client et Aléa serveur



Le résumé de session TLS 1.2

- Le résumé de session peut intervenir pour réutiliser une ancienne session existante (RFC 5705 + maj)
  - On déduit les nouvelles clés des anciennes



Source:https://vincent.bernat.ch/fr/blog/2011-sessions-ssl-rfc5077



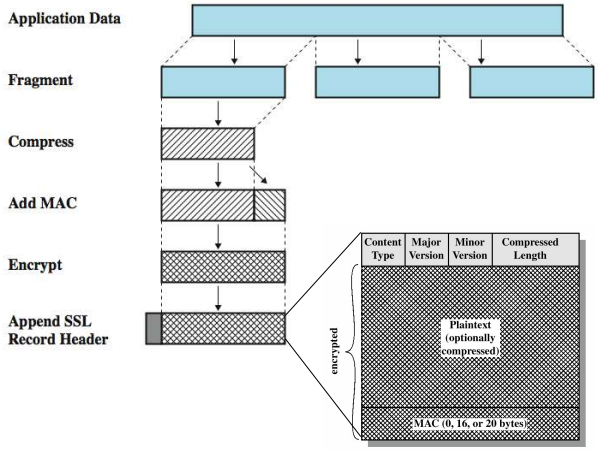
Le résumé de session TLS 1.2

- Pour réaliser ce résumé de session, il existe deux solutions
  - Utilisation d'un identifiant de session dans le message ServerHello
    - Le client le stocke et le présente dans son ClientHello lors du prochain Handshake
    - Si le serveur trouve l'identifiant il passe à la suite, sinon il génère un nouvel identifiant de session et refait le Handshake complet
  - Utilisation d'un « New Session Ticket » dans le handshake
    - Le serveur peut inclure ce ticket dans le dernier échange d'un handshake
    - Ce ticket comprend l'état de la session TLS (secret partagé, algos utilisés) chiffré par une clé connue uniquement du serveur
    - Pour signaler qu'ils supportent tous les deux ce mode, le client envoie un ticket vide dans le ClientHello et le serveur répond avec un ticket vide dans le ServeurHello
- L'utilisation des tickets est préférable car elle limite les ressources allouées coté serveur
  - Il n'a pas besoin de maintenir d'état de connexion entre les sessions notamment



Le protocole TLS Record

- Assure la confidentialité et l'intégrité
- Manipule des fragments d'au plus 2^{14} octets
- Une compression peut également être appliquée si nécessaire



Le protocole TLS Record

- Le protocole TLS Record met également en place des mécanismes de protection de la payload
  - Les fonctions de MAC et de chiffrement permettent d'obtenir un record chiffré
  - Le MAC du record inclus un numéro de séquence pour éviter l'insertion ou le rejeu de message
- Suivant le mode de chiffrement utilisé, la forme du record obtenu pourra être différente
  - Dans le cas de chiffrement par flux (RC4…), le MAC est chiffré en même temps que les données
    - Lorsque le protocole utilisé n'utilise pas de vecteur de synchronisation (comme RC4), l'état du chiffrement par flux de la fin d'un paquet est réutilisé dans le paquet suivant

Le protocole TLS Record

- Pour les chiffrements par bloc (AES…), on utilise
  - des vecteurs d'initialisation (IV)
    - ★ *Un IV doit être aléatoire et imprédictible*
    - ★ *Ces IV ont été ajoutés dans TLS 1.2 pour éviter des attaques sur TLS 1.1*
- Du bourrage (padding) pour envoyer des records de taille homogène



Les améliorations de TLS 1.3

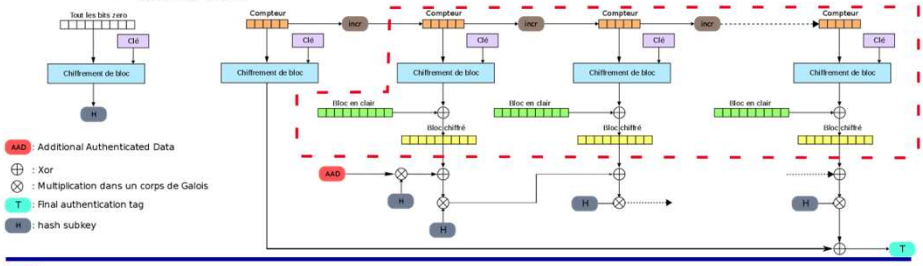
➢ Depuis Aout 2018 TLS 1.3 est standardisé dans la RFC 8446

- La première amélioration notable est l'abandon de toutes les solutions algorithmiques faibles comme RC4, MD4, DSA, SHA-224
  - On ne garde que les algorithmes AEAD (Authenticated Encryption with Associated Data)
- Le format des suite de sécurité repose sur le modèle: TLS_CIPHER_HASH
  - Le mécanisme d'échange de clés n'est plus négociable et repose obligatoirement sur DH éphémère
  - Toutes les suites basées sur RSA ou DH sont supprimées
  - Les fonctions elliptiques sont introduites tout comme EdDSA pour la signature
- On a un nouveau mode 0-RTT pour accélérer l'établissement de connexion
- Tous les messages après le ServeurHello sont maintenant chiffrés et une extension EncryptedExtension est introduite
- Les fonctions de dérivation de clés ont été modifiées pour permettre une analyse de sécurité plus simple par les cryptographes
  - La fonction de dérivation HKDF a été changée en conséquence



TLS Hash - AEAD

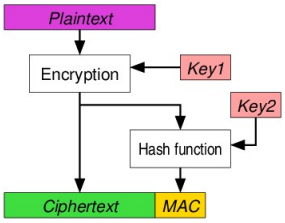
- Dans TLS 1.3 les solutions d'authentification de messages basées sur HMAC sont abandonnées au profit d'AEAD
  - AEAD permet à la fois d'authentifier les données et d'assurer leur sécurité
    - Dans AEAD on vérifie à la fois l'authenticité des données chiffrées (AE) et non chiffrée (AD)
  - Les principaux algorithmes sont AES-GCM (Galois Counter Mode) et Chacha/Poly1305
    - Ex: AES-GCM



37

Retour sur MAC & chiffrement

- Dans TLS 1.3 on utilise le troisième mode qui Encrypt then Mac
  - Utilisé depuis longtemps dans IPSec

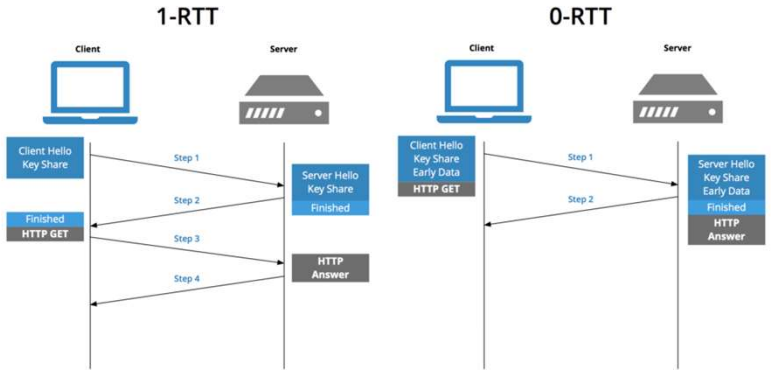


  - On peut utiliser la même clé pour le chiffrement et le MAC mais ceci est fortement déconseillé car cela peut entrainer des problèmes de sécurité

38

TLS 1.3 - 0-RTT

- Le principe est d'éliminer les échanges inutiles en cas de rétablissement de session



Source: https://www.keycdn.com/blog/tls-1-3-support

Les bénéfices de TLS 1.3

- Une meilleure garantie de respect de la vie privée grâce au chiffrement du certificat du serveur
- Une idéologie "une clé par propos" :
  - clés différentes pour le chiffrement des messages du handshake vs. record layer
  - clés différentes pour le HMAC des messages finished
  - clés différentes pour le client vs le serveur
- Chiffrement authentifié sans faire Encrypt then MAC
  - résiste mieux aux attaques Lucky Thirteen
- La dérivation de clé insère le session hash à chaque étape :
  - résiste aux attaques 3Shake, transcript collision, etc.

Quelques attaques protocolaires sur TLS

Présentation générale

- SSL est présent depuis de très nombreuses années et est devenu un standard de fait pour la sécurité des communications IP
- Depuis la version 2, de nombreuses vulnérabilités ont été mises à jour
  - Négociation à la baisse des suites cryptographiques (downgrade attack)
  - Réutilisation de clés pour le chiffrement et le contrôle d'intégrité
  - Utilisation d'algorithmes cryptographiques considérés comme faibles (MD5 par exemple)
  - Cette version est donc considérée comme non sûre et ne doit plus être utilisée
    - Voir la RFC 6176
- La version 3 date de 1996 et est également fortement déconseillée
  - Les piles sont souvent incompatibles avec les avancées de TLS (extensions…)
  - De nombreuses attaques récentes ont montré ses limites



BEAST (Browser Exploit Against SSL/TLS)

- BEAST a été réalisée en 2011 par Doung et Rizzo et fonctionne avec TLS 1.0
  - Elle repose sur une méthode dite du texte en clair choisi (chosen plain-text)
    - L'attaquant génère un texte clair de son choix et le fait chiffrer
    - L'observation du résultat lui permet d'obtenir des informations sur la clé utilisée
  - Cette attaque exploite une faiblesse de TLS1.0 qui consistait à utiliser des vecteurs d'initialisation (IV) implicites
    - Connu depuis 2004 et d'ailleurs corrigée à l'époque dans OpenSSL
    - Corrigé dans TLS 1.1 avec l'utilisation d'IV explicites depuis 2006
- BEAST a pu être réalisée principalement à cause de la lenteur du déploiement des nouvelles versions de TLS dans les navigateurs
  - Les serveurs utilisent en grande majorité OpenSSL comme librairie et ont en général un support de TLS à jour
  - Ce n'est pas vrai pour les navigateurs qui utilisent plutôt NSS
    - En 2011, la plupart des navigateurs ne disposaient pas du support TLS 1.1



BEAST (Browser Exploit Against SSL/TLS)

- Chosen plain-text attack
  - Eve veut connaitre un message M. Elle sait que le premier IV est IV_1 et que le message chiffré par l'oracle de chiffrement est égal à $E_k(M \oplus IV_1)$
  - Si elle connait également IV_2, elle peut alors essayer un message M_{test} et injecter $M_{test} \oplus IV_1 \oplus IV_2$
  - Elle obtient alors $E_k(M_{test} \oplus IV_1 \oplus IV_2 \oplus IV_2) = E_k(M_{test} \oplus IV_1)$
  - Si le message chiffré obtenu est identique à $E_k(M \oplus IV_1)$ alors $M = M_{test}$
  - Dans le même ordre d'idée, si elle connait déjà M cela peut lui permettre de déterminer la clé K
  - Cette attaque a été réalisée lors de la seconde guerre mondiale par les américains pour déterminer un code utilisé par les japonais



BEAST (Browser Exploit Against SSL/TLS)

- Pour réaliser cette attaque, l'attaquant se met en attente d'un cookie de session
  - Facilement détectable dans un échange HTTP
  - Ce cookie va servir d'IV pour le prochain échange chiffré et sa forme en clair est connue
- L'attaquant utilise une approche MIM qui va injecter les segments de textes clairs envoyés par le navigateur cible dans flux chiffré pour déterminer la clé partagée
  - Réalisable avec un script javascript par exemple ou un Web Ad
- L'attaque est assez lente car elle nécessite des sessions d'au moins 30 min pour casser des cookies avec des clés de plus de 1000 caractères



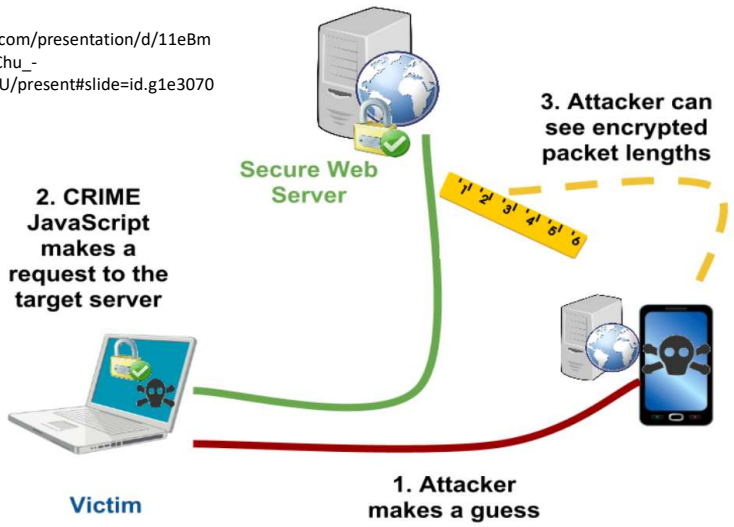
CRIME (Compression Ratio Info-leak Made Easy)

- CRIME a été proposée en 2012 par Duong et Rizzo
  - C'est une attaque de type side channel qui s'appuie sur la compression utilisée dans TLS
    - DEFLATE: LZ77 et codage de Huffman
      - ★ *LZ77 Recherche des répétitions et les remplace par des références*
      - ★ *Huffman: Construit une table qui référence chaque octet avec un code unique*
    - Gzip
- Elle consiste à injecter du texte de taille connue dans les requêtes légitimes et à comparer les tailles de sorties
  - Les algorithmes de compression ont tendance à mieux réduire les éléments contenant des données identiques
    - Si les valeurs testées sont fausses, la taille du message final va avoir tendance à être stable
    - Si la valeur insérée est correcte alors la taille du message en sortie sera plus petite



CRIME (Compression Ratio Info-leak Made Easy)

Source:
https://docs.google.com/presentation/d/11eBmGiHbYcHR9gL5nDyZChu_-lCa2GizeuOfaLU2HOU/present#slide=id.g1e3070b2_2_1



CRIME (Compression Ratio Info-leak Made Easy)

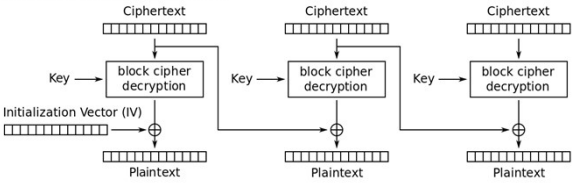
- CRIME peut finalement être évitée en désactivant simplement la compression (qui est optionnelle)
- BREACH (Browser Reconnaissance and Exfiltration via Adaptive Compression of Hypertext) est une version alternative de CRIME qui cible directement HTTP
  - Plus difficile à éviter car la compression HTTP est très utilisée sur le net
  - Solutions:
    - Padding de taille aléatoire
    - Secrets dynamiques
    - Masquage du secret en XORant avec un random
    - …

Lucky Thirteen

- Lucky 13 a été proposée en 2013 par AlFardan et Paterson et cible le mode CBC
  - Le nom vient du chiffre 13
    - Utilisé dans TLS MAC qui utilise des en-têtes de 13octets
    - 13 est une chance pour l'attaquant ;)
  - C'est une variante du Padding Oracle introduit par Vaudenay en 2002
    - L'adversaire soumet un chiffré C en mode CBC à un oracle
      - ★ *S'appuie sur le fait que l'on sait que le bourrage a un format précis*
        - Pour un octet 0x01, pour deux octets 0x02 etc.
      - ★ *L'oracle déchiffre C, vérifie le padding et renvoie VALID ou INVALID (O ou 1)*
    - Permet de déchiffrer les données sans la clé en mode CBC

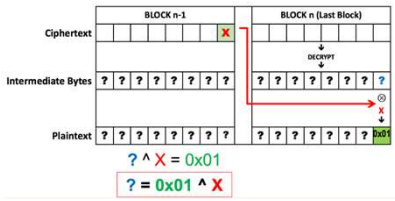


- Rappel du déchiffrement CBC



Cipher Block Chaining (CBC) mode decryption

- Mathématiquement cela revient à $P_i = D_K(C_i) \oplus C_{i-1}, C_0 = IV.$
- Si l'attaquant a deux blocs chiffrés C_1, C_2 , il va modifier le dernier octet de C_1 et interroger l'oracle
  - ★ *Si ok:* $D_K(C_2) = C'_1 \oplus 0\text{x}01$

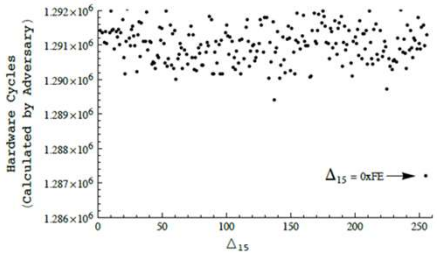


Source: Brian Holyfield, Gotham Digital Science



Lucky Thirteen

- Elle utilise une attaque temporelle sur le MAC de TLS
  - Dans TLS 1.0 les messages d'erreurs générés ont été changés
  - Par contre les différences de temps entre VALID et INVALID sont restés
    - ★ *Les messages contenant au moins deux octets corrects sont traités plus vite que les messages ne contenant pas d'octet correct ou un seul octet correct*
    - ★ *Si le padding est invalide, le MAC n'est pas calculé*
  - Exemple d'attaque et valeur de temps pour obtenir une réponse



Source: http://www.isg.rhul.ac.uk/tls/TLStiming.pdf



POODLE (Padding Oracle on Downgrade Legacy Encryption)

- POODLE est une attaque proposée en 2014 qui vise le protocole TLS Record de SSLv3 et en particulier le padding CBC
  - Ce padding est différent celui de TLS ce qui fait que l'attaque d'origine n'affectait pas TLS 1.2
    - Quelques implémentations de TLS 1.0 et TLS 1.1 ont été touchées
- Dans SSLv3
  - Le padding n'est pas déterministe
    - Seul le dernier octet contient la taille du padding
  - Il n'est pas pris en compte lors du calcul du MAC
    - Il n'est donc pas vérifiable lors du déchiffrement
    - Possibilité de le substituer et d'effectuer un random oracle
- Grâce à cela, il sera possible de déchiffrer un octet de chaque trame et donc de finir par reconstituer un cookie de session



POODLE (Padding Oracle on Downgrade Legacy Encryption)

- Un peu de détail:
  - Le mode CBC nécessite que le message soit d'une taille multiple de la taille de bloc du chiffré
    - 16 pour AES par exemple
  - Pour s'en assurer, SSL ajoute du padding
    - Ce padding peut aller jusqu'à une taille complète de bloc
  - Si on utilise AES et si le dernier bloc ne contient que du padding alors son dernier octet vaut donc 15
  - L'attaquant, qui contrôle la connexion, peut donc prendre un bloc chiffré qu'il souhaite connaître et le substituer à ce bloc de padding
  - Lorsque le serveur reçoit ce paquet, il va essayer d'en déchiffrer le dernier bloc C_n
    - $C_{dechiffre}$ = Dechiffrement(C_n) $\oplus$ C_{n-1}
  - Si $C_{dechiffre}$ contient 15 sur le dernier octet alors il va procéder au déchiffrement du reste du paquet sinon il génère une erreur



POODLE (Padding Oracle on Downgrade Legacy Encryption)

- Si l'attaquant n'observe pas d'erreur il sait donc que la valeur 15 se trouve bien sur le dernier octet du bloc
  - Et surtout que 15 = Dechiffrement(C_n) $\oplus$ C_{n-1}
- A partir de là il est possible de retrouver la valeur de C_n en XORant avec C_{n-1} et avec le dernier octet du bloc précédent le bloc qu'il essaie de retrouver

- Le fait de pouvoir retrouver un bit permet en fait de retrouver tout le message, en particulier, lorsque l'on vise un cookie de session
  - Il suffit de faire varier la taille du message pour aligner les blocs comme on le souhaite
  - Réalisable assez simplement à l'aide d'un script javascript qui injecte du texte dans les Header et les requêtes HTTP
  - On se retrouve dans le cas de l'attaque BEAST



FREAK (Factoring RSA Export Keys)

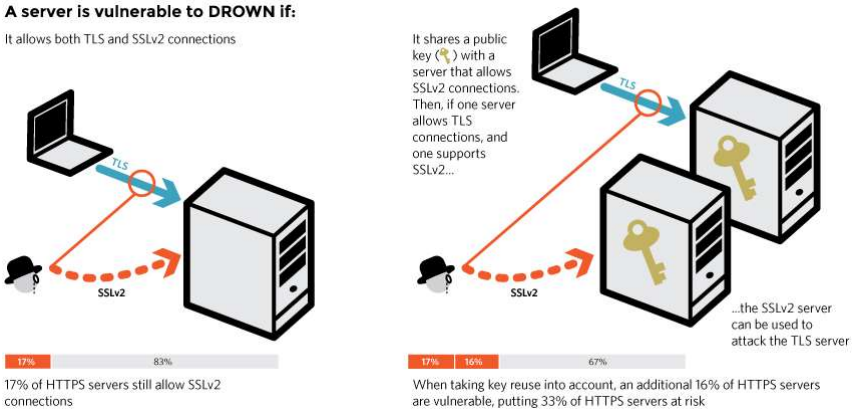
- FREAK a été présentée en 2015 par Bhargavan de l'INRIA
  - Elle permet d'intercepter les communications HTTPS en obligeant le client et le serveur à utiliser des suites cryptographiques faibles
  - Repose sur l'utilisation de clés RSA de 512bits à travers RSA-EXPORT
    - Fonctionnalité imposée par la NSA jusqu'au début des années 2000
    - Consiste à transmettre une clé RSA de 512 avec le Certificat
      - ★ *Cette clé est ensuite utilisée pour chiffrer la pré-master-secret key*
    - Nécessite de négocier une suite EXPORT
      - ★ *Devrait être interdit dans les négociations avec un serveur mais en pratique beaucoup l'autorise encore*
      - ★ *Cette clé devrait être changé périodiquement mais là encore les serveurs ne le font que rarement*
        - La clé est donc valide pour la durée de vie du serveur…
- Cette attaque a fait beaucoup de bruit car présente dans de très nombreuses piles





DROWN (Decrypting RSA with Obsolete and Weakened eNcryption)

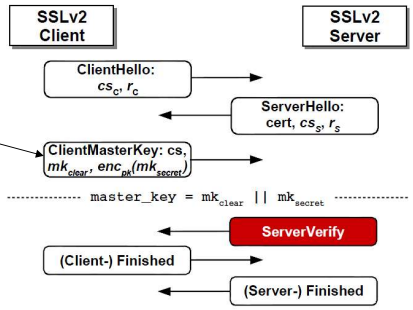
- DROWN est sortie en Mars 2016 et cible les implémentations récentes de TLS
  - S'appuie sur le support de SSLv2 par le serveur et sur le fait que les serveur utilise souvent les même clés RSA quelle que soit la version du protocole utilisée





DROWN (Decrypting RSA with Obsolete and Weakened eNcryption)

- L'idée est en fait de se servir du serveur SSLv2 comme un oracle
  - Plus précisément comme un oracle de Bleichenbacher
- Rappel de SSLv2
  - Le client envoie la PMSK chiffrée
  - Le serveur génère la clé de session
  - Le client envoie Finished chiffré avec la clé de session



- Pour utiliser l'oracle, l'attaquant va générer des suites d'octet aléatoires de la longueur de la PMSK chiffrée
  - Dans la plupart des cas le serveur ne pourra pas déchiffrer et renverra une erreur de padding

DROWN (Decrypting RSA with Obsolete and Weakened eNcryption)

- Le padding avec RSA respecte en effet la norme PKCS #1
  - Cela permet de définir le format du message

00	02	padding string	00	data block

- Dans certains cas cependant, le serveur va réussir à déchiffrer une clé avec un padding correct
  - Dans ce cas, le serveur ne rejettera que le message Finished qui ne sera pas chiffré avec la bonne clé
  - Dans le cas d'SSL, le data block qui nous intéresse contiendra la PMSK
- Cela permet donc à l'attaquant de savoir si une suite d'octets qu'il envoie possède un padding correct ou non
  - Cela lui permet également de savoir que le message démarre par 0x00 0x02

DROWN (Decrypting RSA with Obsolete and Weakened eNcryption)

- Pour en revenir à DROWN
  - L'attaquant va capturer le message *m* contenant la PMSK chiffrée
    - Avec RSA il sait que $m = (PMSK)^e \bmod n$ ou e est l'exposant publique et n la longueur du message (256 pour des clés RSA 2048 par exemple)
  - Il génère une valeur s et envoie au serveur un message $m' = m*s^e \bmod n$
  - Le serveur calcule $PMSK' = (m*s^e)^d \bmod n$ avec d l'exposant privé ce qui équivaut à $PMSK*s$
    - Lorsque la réponse du serveur est bonne cela informe l'attaquant que la clé se trouve dans l'intervalle des entiers démarrant pas 0x00 0x02 ce qui lui permet d'affiner sa recherche
  - Cette information lui permet de raffiner son choix de *s*
  - A partir de là, il génère plusieurs millions de requêtes avec des s différents pour converger vers la valeur de *PMSK*

